

Complexity Analysis

1. Overview

Complexity analysis is the skill of estimating how much **time** and **memory** an algorithm uses as the input size grows.

In placements, online assessments, and competitive programming, complexity analysis helps you answer:

- Will this solution pass for $n = 10^5$?
- Is $O(n^2)$ acceptable here?
- Why is binary search faster than linear search?
- How many recursive calls does this function make?
- Is this nested loop really $O(n^2)$ or something else?

Complexity analysis usually ignores machine-specific details and focuses on growth rate.

Example:

```
for (int i = 0; i < n; i++) {  
    cout << i << "\n";  
}
```

This loop runs n times, so its time complexity is $O(n)$.

2. Intuition

Think of complexity as describing the **shape of growth**.

If input size doubles:

Complexity	Work for n	Work for $2n$	Growth Feeling
$O(1)$	1	1	No change
$O(\log n)$	about 17 for 10^5	about 18	Very slow growth
$O(n)$	n	$2n$	Doubles
$O(n \log n)$	$n \log n$	a bit more than doubles	Sorting-like
$O(n^2)$	n^2	$4n^2$	Becomes expensive fast
$O(2^n)$	exponential	explodes	Usually impossible for large n

Simple analogy:

- $O(1)$: opening a bookmarked page.
- $O(\log n)$: finding a word in a dictionary by repeatedly halving pages.
- $O(n)$: checking every student in a class.

- $O(n^2)$: comparing every student with every other student.
- $O(2^n)$: trying every possible subset.

The main idea:

1. Count how many times important operations run.
2. Keep the fastest-growing term.
3. Drop constants.
4. Express the result using Big-O, Big-Theta, or Big-Omega.

Why this works:

For large inputs, growth rate matters more than exact operation count.

Example:

$$3n^2 + 10n + 500$$

For large n , the n^2 term dominates. So the complexity is $O(n^2)$.

3. When to Use It

Use complexity analysis whenever you need to judge whether an algorithm is efficient enough.

Common situations:

- Before submitting a solution in CP or online assessment.
- While comparing brute force and optimized approaches.
- During interviews when explaining why your solution is efficient.
- When constraints are given, such as $n \leq 10^5$.
- When debugging TLE or MLE.
- When analyzing recursion, DP, graph traversal, sorting, searching, or data structures.

Common trigger phrases:

- "Find an efficient algorithm"
- "Optimize the brute force"
- "Expected time complexity"
- "Can this pass for 10^5 ?"
- "Analyze the complexity"
- "Worst-case time"
- "Memory limit exceeded"
- "Time limit exceeded"
- "Nested loops"
- "Recursive relation"
- "Divide and conquer"

4. When Not to Use It

Do not over-focus on complexity when:

- Input size is tiny and brute force is clearly enough.
- The problem asks for simple simulation with small constraints.
- Constant factors dominate in practical performance.
- The bottleneck is I/O, not algorithmic growth.
- You are comparing two algorithms with the same Big-O but very different implementation costs.

Simpler alternatives:

- For $n \leq 20$, exponential backtracking may be intended.
- For $n \leq 100$, $O(n^3)$ may pass.
- For $n \leq 10^5$, usually target $O(n)$, $O(n \log n)$, or sometimes $O(\sqrt{n})$.

Common wrong assumptions:

- Every nested loop is $O(n^2)$. Some are $O(n \log n)$ or $O(n)$.
- Every recursive function is exponential. Some are logarithmic or linear.
- Hash maps are always $O(1)$. Worst case can degrade, though average case is usually $O(1)$.
- $O(n \log n)$ is always fine. For very large n or tight time limits, constants still matter.

5. Core Concepts

Big-O Notation

Big-O gives an **upper bound** on growth.

If an algorithm is $O(n^2)$, it means its running time does not grow faster than some constant multiple of n^2 for large enough n .

Why it matters:

Interviewers usually ask for Big-O because it describes the worst useful bound.

Example:

```
for (int i = 0; i < n; i++) {}
```

This is $O(n)$.

Big-Theta Notation

Big-Theta gives a **tight bound**.

If an algorithm is $\Theta(n)$, it grows exactly like n up to constant factors.

Example:

```
for (int i = 0; i < n; i++) {}
```

This is $\Theta(n)$ because it always runs exactly n times.

Big-Omega Notation

Big-Omega gives a **lower bound**.

If an algorithm is $\Omega(n)$, it takes at least linear time for large enough input.

Example:

Finding an element in an unsorted array has worst-case $\Omega(n)$ because sometimes you must inspect every element.

Time Complexity

Time complexity measures how the number of operations grows with input size.

Important things to count:

- Loop iterations
- Recursive calls
- Data structure operations
- Sorting
- Graph traversals
- DP state transitions

Space Complexity

Space complexity measures extra memory used by the algorithm.

Count:

- Arrays, vectors, maps, sets
- Recursion stack
- DP tables
- Graph adjacency lists
- Queues and stacks

Usually, input storage is not counted unless the problem asks for total memory.

Worst, Average, and Best Case

Case	Meaning	Example: Linear Search
Best case	Minimum work	Target is first element: $O(1)$
Average case	Expected work	Target around middle: $O(n)$
Worst case	Maximum work	Target absent or last: $O(n)$

Interviews usually expect worst-case unless stated otherwise.

Amortized Analysis

Amortized analysis gives average cost per operation over a sequence of operations.

It is useful when one operation is sometimes expensive but most operations are cheap.

Example:

`vector.push_back()` is amortized $O(1)$ because resizing is rare.

Recursion Complexity

Recursive complexity is usually described using a recurrence.

Example:

$$T(n) = T(n / 2) + O(1)$$

This means the function solves one half-size subproblem and does constant extra work. Complexity is $O(\log n)$.

Master Theorem Basics

Master theorem solves recurrences of the form:

$$T(n) = aT(n / b) + f(n)$$

Where:

- a = number of recursive subproblems
- n / b = size of each subproblem
- $f(n)$ = work done outside recursion

Compare $f(n)$ with $n^{(\log_b a)}$.

Case	Condition	Result
1	$f(n)$ smaller than $n^{(\log_b a)}$	$\Theta(n^{(\log_b a)})$
2	$f(n)$ same as $n^{(\log_b a)}$	$\Theta(n^{(\log_b a)} \log n)$
3	$f(n)$ larger than $n^{(\log_b a)}$	$\Theta(f(n))$

Examples:

Recurrence	Complexity	Common Algorithm
$T(n) = T(n/2) + O(1)$	$O(\log n)$	Binary search
$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$	Merge sort
$T(n) = 2T(n/2) + O(1)$	$O(n)$	Full binary recursion

Recurrence	Complexity	Common Algorithm
$T(n) = T(n/2) + O(n)$	$O(n)$	Some divide-and-conquer scans

Recurrence Solving

Common methods:

- Substitution
- Recursion tree
- Master theorem
- Repeated expansion

Example:

$$T(n) = T(n - 1) + O(1)$$

Expanding:

$$\begin{aligned} T(n) &= T(n - 1) + 1 \\ T(n - 1) &= T(n - 2) + 1 \\ T(n - 2) &= T(n - 3) + 1 \\ &\dots \end{aligned}$$

After n steps, total work is $O(n)$.

Logarithmic Complexity Intuition

Logarithmic complexity appears when the search space repeatedly shrinks by a constant factor.

Example:

```
for (int i = 1; i <= n; i *= 2) {}
```

Values of i :

1, 2, 4, 8, 16, ...

After k iterations:

$$i = 2^k$$

Loop stops when:

$$2^k > n$$

So:

$$k = \log_2(n)$$

Complexity: $O(\log n)$.

Potential Method for Amortized Analysis

Potential method assigns a "stored credit" to the data structure.

Idea:

- Cheap operations may save credit.
- Expensive operations spend credit.
- Over many operations, average cost remains small.

Example:

Dynamic array resizing:

- Normal push: cost $O(1)$.
- Resize push: cost $O(n)$ because elements are copied.
- But resizing happens after many cheap pushes.
- Stored credit from previous pushes pays for copying.

Therefore, each push is amortized $O(1)$.

6. Step-by-Step Algorithm

Use this algorithm to analyze most code quickly:

1. Identify the input size: usually n , sometimes m , V , E , rows , cols , or string length.
2. Count simple loops.
3. For nested loops, calculate total iterations, not just nesting depth.
4. For loops that multiply or divide the variable, use logarithms.
5. For recursion, write a recurrence.
6. For data structures, include operation costs.
7. For sorting, add $O(n \log n)$.
8. For graph algorithms, express complexity using V and E .
9. Drop constants.
10. Keep only the dominant term.
11. Analyze space separately.
12. Mention worst, average, and best case when behavior depends on input.

Quick constraint guide:

Constraint	Usually Acceptable
$n \leq 10$	$O(n!)$, $O(2^n)$ sometimes
$n \leq 20$	$O(2^n)$
$n \leq 100$	$O(n^3)$
$n \leq 1000$	$O(n^2)$
$n \leq 10^5$	$O(n \log n)$ or $O(n)$
$n \leq 10^6$	$O(n)$
$n \geq 10^9$	$O(\log n)$ or $O(1)$

7. Dry Run

Example 1: Multiplicative Loop

Code:

```
for (int i = 1; i <= n; i *= 2) {}
```

Let $n = 32$.

Iteration	i
1	1
2	2
3	4
4	8
5	16
6	32
Stop	64

The loop runs 6 times.

Since $32 = 2^5$, the number of iterations is about $\log_2(n) + 1$.

Complexity: $O(\log n)$.

Example 2: Triangular Nested Loop

Code:

```
for (int i = 0; i < n; i++)
    for (int j = i; j < n; j++) {}
```

Let $n = 5$.

i	Values of j	Count
0	0, 1, 2, 3, 4	5
1	1, 2, 3, 4	4
2	2, 3, 4	3
3	3, 4	2
4	4	1

Total iterations:

$$5 + 4 + 3 + 2 + 1 = 15$$

For general n :

$$n + (n - 1) + (n - 2) + \dots + 1 = n(n + 1) / 2$$

Drop constants and lower terms:

$$\Theta(n^2)$$

Complexity: $O(n^2)$.

8. C++ Implementation

Complexity analysis is a thinking skill, not a normal algorithm with one fixed implementation. The code below is a small reusable C++17 helper program that prints common iteration counts and demonstrates how to verify loop behavior.

```
#include <bits/stdc++.h>
using namespace std;

long long countLinearLoop(long long n) {
    long long operations = 0;

    for (long long i = 0; i < n; i++) {
```

```
        operations++;
    }

    return operations; // n
}

long long countLogLoop(long long n) {
    long long operations = 0;

    for (long long i = 1; i <= n; i *= 2) {
        operations++;
    }

    return operations; // floor(log2(n)) + 1 for n >= 1
}

long long countTriangularNestedLoop(long long n) {
    long long operations = 0;

    for (long long i = 0; i < n; i++) {
        for (long long j = i; j < n; j++) {
            operations++;
        }
    }

    return operations; // n * (n + 1) / 2
}

long long countQuadraticLoop(long long n) {
    long long operations = 0;

    for (long long i = 0; i < n; i++) {
        for (long long j = 0; j < n; j++) {
            operations++;
        }
    }

    return operations; // n * n
}

string likelyComplexityFromConstraints(long long n) {
    if (n <= 10) return "O(n!), O(2^n), O(n^3), O(n^2), O(n log n), O(n)";
    if (n <= 20) return "O(2^n), O(n^3), O(n^2), O(n log n), O(n)";
    if (n <= 100) return "O(n^3), O(n^2), O(n log n), O(n)";
    if (n <= 1000) return "O(n^2), O(n log n), O(n)";
    if (n <= 100000) return "O(n log n), O(n)";
    if (n <= 1000000) return "O(n)";
    return "O(log n) or O(1)";
}

int main() {
    long long n;
    cin >> n;
```

```

    cout << "n = " << n << "\n";
    cout << "Linear loop operations: " << countLinearLoop(n) << "\n";
    cout << "Logarithmic loop operations: " << countLogLoop(n) << "\n";
    cout << "Triangular nested loop operations: " << countTriangularNestedLoop(n)
<< "\n";
    cout << "Quadratic loop operations: " << countQuadraticLoop(n) << "\n";
    cout << "Usually acceptable complexities: " <<
likelyComplexityFromConstraints(n) << "\n";

    return 0;
}

```

Example input:

5

Example output:

```

n = 5
Linear loop operations: 5
Logarithmic loop operations: 3
Triangular nested loop operations: 15
Quadratic loop operations: 25
Usually acceptable complexities: O(n!), O(2^n), O(n^3), O(n^2), O(n log n), O(n)

```

pYTHON IMPLEMENTATION

pROVIDE CLEAN PYTHON CODE

```

def count_linear_loop(n: int) -> int:
    operations = 0

    for _ in range(n):
        operations += 1

    return operations

def count_log_loop(n: int) -> int:
    operations = 0
    i = 1

    while i <= n:
        operations += 1
        i *= 2

    return operations

```

```
def count_triangular_nested_loop(n: int) -> int:
    operations = 0

    for i in range(n):
        for _ in range(i, n):
            operations += 1

    return operations

def count_quadratic_loop(n: int) -> int:
    operations = 0

    for _ in range(n):
        for _ in range(n):
            operations += 1

    return operations

def likely_complexity_from_constraints(n: int) -> str:
    if n <= 10:
        return "O(n!), O(2^n), O(n^3), O(n^2), O(n log n), O(n)"
    if n <= 20:
        return "O(2^n), O(n^3), O(n^2), O(n log n), O(n)"
    if n <= 100:
        return "O(n^3), O(n^2), O(n log n), O(n)"
    if n <= 1000:
        return "O(n^2), O(n log n), O(n)"
    if n <= 100000:
        return "O(n log n), O(n)"
    if n <= 1000000:
        return "O(n)"
    return "O(log n) or O(1)"

def main() -> None:
    n = int(input())

    print(f"n = {n}")
    print(f"Linear loop operations: {count_linear_loop(n)}")
    print(f"Logarithmic loop operations: {count_log_loop(n)}")
    print(f"Triangular nested loop operations: {count_triangular_nested_loop(n)}")
    print(f"Quadratic loop operations: {count_quadratic_loop(n)}")
    print(f"Usually acceptable complexities: {likely_complexity_from_constraints(n)}")

if __name__ == "__main__":
    main()
```

10. Code Explanation

countLinearLoop

```
for (long long i = 0; i < n; i++)
```

The loop starts at `0`, increments by `1`, and stops before `n`.

Number of iterations:

`n`

Complexity: $\Theta(n)$.

countLogLoop

```
for (long long i = 1; i <= n; i *= 2)
```

The variable doubles each time.

Values:

`1, 2, 4, 8, ...`

After `k` iterations:

`i = 2k`

So the number of iterations is logarithmic.

Complexity: $\Theta(\log n)$.

Important edge case:

If `i` starts at `0`, `i *= 2` never changes `i`, causing an infinite loop.

countTriangularNestedLoop

```
for (long long i = 0; i < n; i++) {  
    for (long long j = i; j < n; j++) {  
        operations++;  
    }  
}
```

```
}
}
```

The inner loop does not always run n times.

It runs:

$n, n - 1, n - 2, \dots, 1$

Total:

$n(n + 1) / 2$

Complexity: $\Theta(n^2)$.

countQuadraticLoop

Both loops run n times.

Total:

$n * n = n^2$

Complexity: $\Theta(n^2)$.

likelyComplexityFromConstraints

This function maps input constraints to likely acceptable complexities. It is not a mathematical proof, but it is very useful in contests and interviews.

11. Complexity Analysis

Code / Concept	Time Complexity	Space Complexity	Notes
Single loop over n	$O(n)$	$O(1)$	Linear scan
Two independent loops	$O(n + m)$	$O(1)$	Keep both if variables differ
Two sequential loops over n	$O(n)$	$O(1)$	$O(2n)$ becomes $O(n)$
Nested $n \times n$ loops	$O(n^2)$	$O(1)$	Classic pair checking
Triangular nested loop	$O(n^2)$	$O(1)$	$n(n+1)/2$
Multiplicative loop $i *= 2$	$O(\log n)$	$O(1)$	Base does not matter in Big-O
Dividing loop $n /= 2$	$O(\log n)$	$O(1)$	Binary-search style

Code / Concept	Time Complexity	Space Complexity	Notes
Sorting	$O(n \log n)$	Depends	<code>std::sort</code> uses introsort
Binary search	$O(\log n)$	$O(1)$ iterative	Requires sorted search space
DFS/BFS adjacency list	$O(V + E)$	$O(V)$	Plus graph storage
Hash map operations	Average $O(1)$	$O(n)$	Worst case can degrade
Balanced BST operations	$O(\log n)$	$O(n)$	<code>set</code> , <code>map</code> in C++
DP table $n \times m$	$O(nm)$	$O(nm)$	Can sometimes optimize space
Recursion depth n	Depends	$O(n)$	Stack memory matters

Preprocessing, query, and update examples:

Technique	Preprocessing	Query	Update	Space
Prefix sum	$O(n)$	$O(1)$	$O(n)$	$O(n)$
Fenwick tree	$O(n \log n)$ or $O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Segment tree	$O(n)$	$O(\log n)$	$O(\log n)$	$O(n)$
Sparse table	$O(n \log n)$	$O(1)$ for idempotent ops	Not suitable	$O(n \log n)$

12. Common Patterns

Pattern Name	How to Identify It	General Approach	Example Problems
Linear scan	Need to inspect all elements once	$O(n)$ loop	Maximum Subarray, Contains Duplicate
Pair checking	Need all pairs	Nested loops, maybe optimize with hash/sort	Two Sum, 3Sum
Shrinking search space	Sorted array or monotonic answer	Binary search, $O(\log n)$ or $O(n \log \text{answer})$	Binary Search, Aggressive Cows
Sorting first	Need order, grouping, intervals	Sort then scan	Merge Intervals, Meeting Rooms
Sliding window	Contiguous subarray/string	Maintain left and right pointers	Longest Substring Without Repeating Characters
Prefix sums	Many range sum queries	Precompute cumulative sums	Range Sum Query, Subarray Sum Equals K
Graph traversal	Nodes and edges	BFS/DFS: $O(V + E)$	Number of Islands, CSES Building Roads
DP states	Overlapping subproblems	Count states times transition cost	Coin Change, Longest Common Subsequence

Pattern Name	How to Identify It	General Approach	Example Problems
Backtracking	Generate choices/combinations	Usually exponential	Subsets, N-Queens
Divide and conquer	Split into subproblems	Write recurrence	Merge Sort, Quick Sort

13. Common Mistakes

- Calling every nested loop $O(n^2)$ without counting actual iterations.
- Forgetting that `for (i = 1; i <= n; i *= 2)` is $O(\log n)$.
- Starting a multiplicative loop from 0, causing an infinite loop.
- Ignoring recursion stack in space complexity.
- Saying binary search is $O(\log n)$ when the search space is not sorted or monotonic.
- Forgetting sorting cost before a linear scan.
- Treating `unordered_map` as guaranteed worst-case $O(1)$.
- Dropping different variables incorrectly: $O(n + m)$ should not become $O(n)$ unless m is bounded by n .
- Ignoring input constraints.
- Confusing $O(n)$ with $\Theta(n)$.
- Forgetting that $O(2^n)$ may pass for $n \leq 20$ but not for $n = 50$.
- Using `int` for operation counts like $n * n$ when n can be large.
- Missing memory limits when using large DP arrays.
- Assuming best case is enough for interviews; worst case is usually expected.

14. Edge Cases

Complexity analysis edge cases to think about:

- $n = 0$
- $n = 1$
- Very large n
- Multiple input variables, such as n and m
- Sparse graph: E much smaller than V^2
- Dense graph: E close to V^2
- Recursion depth too large
- Empty input
- Single element
- All equal elements
- Already sorted input
- Reverse sorted input
- Duplicate values
- Negative values
- Large values causing overflow
- Disconnected graph
- Cycles in graph recursion
- Hash collisions in theoretical worst case
- Algorithms with expensive hidden operations, such as copying strings or vectors

15. Variations

Variation	What Changes	When Used	Placement / CP Importance
Worst-case analysis	Maximum possible operations	Interviews, guarantees	Very high
Average-case analysis	Expected operations over typical inputs	Randomized algorithms, hashing	Medium-high
Best-case analysis	Minimum possible operations	Early exit algorithms	Medium
Amortized analysis	Average over operation sequence	Dynamic arrays, stacks, queues	High
Recurrence analysis	Recursive running time	Divide and conquer, DP	Very high
Space analysis	Memory growth	DP, recursion, graphs	Very high
Constraint-based analysis	Infer needed complexity from limits	CP and online assessments	Very high
Probabilistic analysis	Expected behavior with randomness	Randomized quicksort, hashing	Medium

16. Related Algorithms/Data Structures

- Arrays and Strings: usually analyzed with scans, nested loops, prefix sums, and sliding windows.
- Hash Map vs Balanced BST: hash map gives average $O(1)$, BST gives guaranteed $O(\log n)$ with sorted order.
- Binary Search: classic $O(\log n)$ pattern over sorted or monotonic spaces.
- Sorting: often costs $O(n \log n)$ and enables easier linear solutions afterward.
- Recursion and Divide and Conquer: analyzed using recurrence relations.
- Dynamic Programming: complexity equals number of states times transition cost.
- Graph Algorithms: BFS/DFS are usually $O(V + E)$, Dijkstra is often $O((V + E) \log V)$.
- Segment Tree vs Fenwick Tree: both support logarithmic updates and queries, but segment tree is more flexible.
- Heap vs Balanced BST: heap gives fast min/max access, BST supports ordered search and deletion.
- BFS vs Dijkstra: BFS works for unweighted shortest path; Dijkstra handles non-negative weighted edges.
- DFS vs DSU: DFS explores connectivity directly; DSU handles repeated union/find connectivity queries.
- KMP vs Z Algorithm: both analyze strings in linear time.

17. Practice Problems

Easy

Problem	Platform	Main Idea / Pattern	Difficulty
Running Sum of 1d Array	LeetCode	Linear scan and prefix sum complexity	Easy

Problem	Platform	Main Idea / Pattern	Difficulty
Binary Search	LeetCode	Logarithmic complexity	Easy

Medium

Problem	Platform	Main Idea / Pattern	Difficulty
3Sum	LeetCode	Compare $O(n^3)$ brute force with $O(n^2)$ sorting + two pointers	Medium
Subarray Sum Equals K	LeetCode	Prefix sum + hash map, average $O(n)$	Medium
Number of Islands	LeetCode	DFS/BFS complexity $O(\text{rows} * \text{cols})$	Medium

Hard

Problem	Platform	Main Idea / Pattern	Difficulty
Merge k Sorted Lists	LeetCode	Heap complexity $O(N \log k)$	Hard
Edit Distance	LeetCode	DP states $O(nm)$	Hard
Distinct Subsequences	LeetCode	DP transition and space optimization	Hard

18. Interview Explanation

Complexity analysis tells us how an algorithm's running time and memory usage grow with input size. I usually count the dominant operations, such as loop iterations, recursive calls, data structure operations, or DP transitions. Then I drop constants and lower-order terms to express the growth using Big-O. For recursive algorithms, I write a recurrence like $T(n) = 2T(n/2) + O(n)$ and solve it using recursion tree intuition or the Master theorem. I also mention space separately, including extra arrays and recursion stack. In interviews, I generally give worst-case complexity unless average or best case is specifically relevant.

19. Revision Notes

- Big-O: upper bound.
- Big-Theta: tight bound.
- Big-Omega: lower bound.
- Single loop: $O(n)$.
- Nested full loops: $O(n^2)$.
- Triangular nested loop: $n(n + 1) / 2 = O(n^2)$.
- Multiplying or dividing loop variable by constant: $O(\log n)$.
- Sorting: $O(n \log n)$.
- BFS/DFS with adjacency list: $O(V + E)$.
- DP complexity: number of states times transition cost.
- Recursion space includes call stack.
- Master theorem form: $T(n) = aT(n/b) + f(n)$.
- Binary search needs sorted or monotonic search space.

- Amortized $O(1)$ means average over many operations, not every operation is constant.
- Common trap: different variables should stay different, like $O(n + m)$.
- Common trap: hidden copying can turn clean-looking code expensive.

20. Final Cheat Sheet

Situation	Complexity
Access array element by index	$O(1)$
Scan array once	$O(n)$
Scan two arrays separately	$O(n + m)$
Full nested loops over same n	$O(n^2)$
<code>for (i = 1; i <= n; i *= 2)</code>	$O(\log n)$
<code>while (n > 0) n /= 2</code>	$O(\log n)$
Sort array	$O(n \log n)$
Binary search	$O(\log n)$
Generate all subsets	$O(2^n)$
Generate all permutations	$O(n!)$
DFS/BFS graph	$O(V + E)$

When to use:

- Use complexity analysis before choosing or submitting an approach.
- Use constraints to guess required complexity.
- Use recurrence analysis for recursion.
- Use amortized analysis for data structures with occasional expensive operations.

Main operations:

- Count loops.
- Sum nested loop iterations.
- Convert halving/doubling to logarithms.
- Count recursive branches and depth.
- Include data structure costs.
- Analyze space separately.

Key code ideas:

```
for (int i = 1; i <= n; i *= 2) {}
// O(log n)
```

```
for (int i = 0; i < n; i++)  
    for (int j = i; j < n; j++) {}  
// O(n^2), because total iterations = n + (n - 1) + ... + 1
```

Important edge cases:

- $n = 0$ or $n = 1$
- Very large n
- Multiple variables: n, m, V, E
- Recursion depth
- Memory limits
- Hidden expensive operations
- Infinite loops with bad update logic